VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF ELECTRICAL & ELECTRONICS ENGINEERING

----------o0o----------

EMBEDDED SYSTEM DESIGN

- PROJECT -

DIGITAL CLOCK WITH ALARM

Instructor: MSc. Bui Quoc Bao

Semester: 252

Class: CC01

Group: 6

No.	Student Name	Contribution	Student ID
1	Dương Tuấn Hưng	100%	2351035
2	Trương Tấn Đạt	100%	2351017
3	Bùi Bách Tùng	100%	2351073

Ho Chi Minh City, May 2026

TABLE OF CONTENT

LIST OF FIGURES

LIST OF TABLES

1 INTRODUCTION

Embedded systems are commonly used in small real-time products where the hardware and firmware must work together under limited resources. A digital clock with alarm is a suitable project for this course because it combines several basic but important embedded functions: real-time data acquisition, display control, button-based user input, non-volatile storage, and output control through a buzzer.

In this project, the group designed and implemented a digital clock system using an STM32 microcontroller, an RTC module, an LCD1602 display, push buttons, and a buzzer. The RTC keeps track of time, while the STM32 handles the user interface, alarm logic, LCD update, and alarm output.

The goal of the project was not only to make the clock display time, but also to build a complete embedded prototype that can be configured by the user. The system supports time setting, alarm setting, alarm triggering, and alarm data retention after reset or power loss.

The implementation was verified mainly on a breadboard prototype. A PCB layout was also designed and checked, but the board was not fabricated in this phase. Therefore, the project focuses on functional verification at prototype level rather than final product assembly.

2 SYSTEM REQUIREMENTS

2.1 Overview

This section defines the system requirements for the digital clock and alarm project. The requirements are divided into functional requirements, non-functional requirements, and design constraints. Level-1 requirements describe the main expected functions or quality goals of the system. Level-2 requirements give more specific details and measurable values where applicable.

The system is built around the STM32F103C8T6 microcontroller. The MCU controls the LCD1602 display, reads the four push buttons, communicates with the external DS3231 real-time clock, controls the buzzer, and stores alarm settings in internal flash memory. The STM32F103C8T6 platform and its internal flash/peripheral behavior are documented by STMicroelectronics [1], [2]. The DS3231 provides external real-time clock capability through

an I2C-compatible interface [3], [4]. The LCD1602 display uses an HD44780-compatible character LCD interface [5].

The requirements in this section are written based on the implemented project architecture: STM32F103C8T6, LCD1602, DS3231 RTC, four push buttons, buzzer output, software I2C communication, and internal flash alarm storage.

2.2 Functional requirements

Functional requirements describe what the system shall do. In this project, the main functions are real-time clock display, time/date setting, alarm operation, and non-volatile alarm storage. Table 2.1 summarizes the functional requirements at level 1 and level 2.

Table 2.1: Functional requirements of the digital clock and alarm system.

ID	Requirement statement
FR1	The system shall display real-time clock information.
FR1.1	The system shall display the current hour, minute, second, date, month, year, and day-of-week on the LCD1602.
FR1.2	The system shall show alarm status and RTC temperature when these values are available. It shall also show RTC or storage error messages when a fault is detected.
FR2	The system shall allow the user to configure the current time and date.
FR2.1	The SET_TIME button shall step through hour, minute, second, day-of-week, date, month, year, and finish fields.
FR2.2	The UP and DOWN buttons shall adjust the selected field, and the firmware shall write the final date/time values to the external RTC after the setting process is finished.
FR3	The system shall provide alarm configuration and alarm output.
FR3.1	The system shall support three alarm slots. Each alarm shall include enable status, hour, minute, and second values.
FR3.2	When the current time matches an enabled alarm, the buzzer shall be activated with a 300 ms ON and 100 ms OFF pattern for a total duration of 60 seconds.
FR4	The system shall preserve alarm settings after power loss.
FR4.1	The firmware shall save all alarm settings into the STM32 internal flash memory after alarm configuration is completed.
FR4.2	At startup, the firmware shall load the latest valid alarm record from flash. If no valid record exists, it shall use default alarm values.

2.3 Non-functional requirements

Non-functional requirements describe how well the system shall operate. These requirements focus on response time, usability, reliability, and maintainability. Table 2.2 lists the main non-functional requirements for the project.

Table 2.2: Non-functional requirements of the digital clock and alarm system.

ID	Requirement statement
NFR1	The system shall detect button presses correctly and update the display without noticeable delay.
NFR1.1	Button state changes shall be accepted only after a 10 ms debounce interval to reduce false triggering caused by mechanical bouncing.
NFR1.2	The UP and DOWN buttons shall support repeated adjustment every 100 ms while the button is held.
NFR2	The system shall provide a readable and clear user interface.
NFR2.1	The LCD content shall be refreshed every 50 ms during runtime to keep the displayed information synchronized with firmware variables.
NFR2.2	The selected editing field shall blink every 300 ms so the user can identify which value is being modified.
NFR3	The system shall improve data reliability for alarm storage.
NFR3.1	The non-volatile storage structure shall use a magic value, sequence number, checksum, and commit marker to verify the alarm data saved in internal flash.
NFR3.2	After saving alarms, the firmware shall load the saved data back and compare it with the original RAM data before accepting the save operation.
NFR4	The firmware shall be maintainable and modular.
NFR4.1	The firmware shall separate application logic and hardware-related drivers into different modules, such as app_clock, bsp_buttons, bsp_lcd, bsp_rtc, and bsp_nv.
NFR4.2	Project constants such as pin mapping, timing values, alarm count, RTC address, and flash storage settings shall be centralized in the configuration header file.
NFR5	The system shall be designed with a reasonable implementation cost.
NFR5.1	The total cost of the main electronic components shall be kept within an estimated budget of 300,000–500,000 VND for one prototype.
NFR5.2	The design shall use commonly available components such as STM32F103C8T6, LCD1602, DS3231 RTC module, push buttons, buzzer, and standard power components to reduce purchasing and replacement cost.

2.4 Design constraints

Design constraints are fixed conditions that guide the implementation. These constraints come from the selected components, the pin assignment, the stable firmware branch, and the

available hardware resources. Table 2.3 summarizes the main design constraints used in this project.

Table 2.3: Design constraints of the digital clock and alarm system.

ID	Constraint area	Constraint description
DC1	Microcontroller	The design shall use the STM32F103C8T6 Blue Pill platform as the main controller [1], [2].
DC2	Display	The design shall use a 16x2 LCD1602 display with an HD44780-compatible 4-bit interface [5].
DC3	RTC module	The design shall use an external DS3231 RTC module for timekeeping and temperature reading [3].
DC4	RTC communication	The final firmware shall use software I2C on PB6 as SCL and PB7 as SDA. The I2C bus concept is based on SDA and SCL serial communication lines [4].
DC5	User input	The design shall use four active-low push buttons: SET_TIME on PA0, UP on PA1, DOWN on PA2, and SET_ALARM on PA3.
DC6	Alarm output	The buzzer output shall be controlled through PA8. The firmware shall generate the buzzer timing pattern by GPIO control.
DC7	Alarm storage	Alarm settings shall be stored in STM32 internal flash memory. The implemented firmware reserves the last 1 KB flash page starting at 0x0800FC00 for non-volatile alarm storage.
DC8	Firmware architecture	The firmware shall follow the current modular structure. The main loop shall call APP_CLOCK_Task(), while hardware-specific functions remain inside BSP driver modules.

These requirements and constraints describe what the system must do and what limits the design must follow. They also help with later testing, because each main hardware and firmware function can be compared with its corresponding requirement.

3 SYSTEM ARCHITECTURE

3.1 Block Diagram

Figure 3.1 presents the overall block diagram of the digital clock and alarm system. The diagram summarizes the main subsystems of the project, including the power supply, STM32F103C8T6 microcontroller, push-button input, LCD display, real-time clock

subsystem, non-volatile alarm storage, and buzzer output. The STM32F103C8T6 is used as the central controller because it provides GPIO, embedded flash memory, and clock resources required for this design [1], [2].

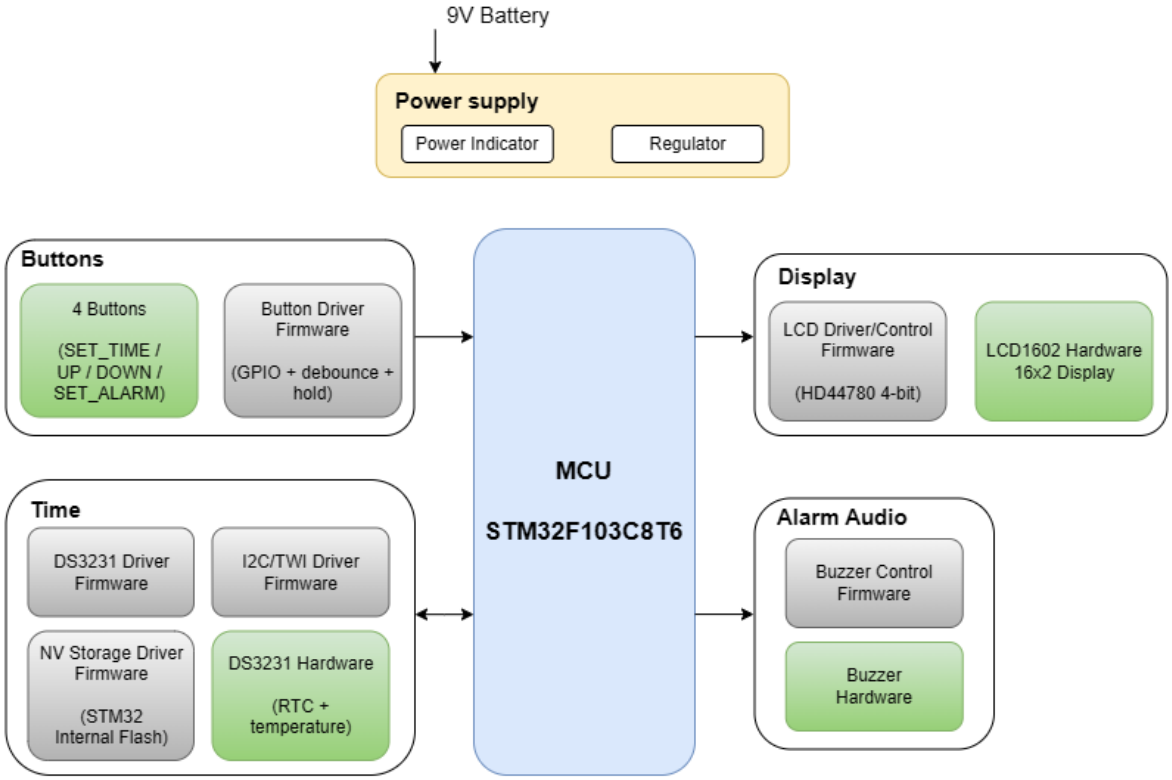


Figure 3.1: System block diagram of the digital clock and alarm system.

The power supply block receives power from a 9 V battery. This input is regulated before being supplied to the rest of the circuit. The power indicator provides a simple visual signal that the board is powered. This block supports stable operation of the MCU and all external modules.

The MCU block is the main processing unit of the system. It executes the firmware, reads the user buttons, communicates with the RTC module, updates the LCD, manages alarm settings, controls the buzzer, and stores alarm data in internal flash memory. Therefore, all other functional blocks are coordinated through the STM32F103C8T6.

The button block provides the user input interface. The system uses four buttons: SET_TIME, UP, DOWN, and SET_ALARM. These buttons are connected to the MCU and processed by a firmware button driver. The driver reads the GPIO state, applies debounce

processing, detects press events, and detects held buttons. This allows the user to set the current time, change alarm values, and move through configuration fields.

The display block provides the visual output of the system. It includes the LCD1602 hardware module and the LCD driver implemented in firmware. The LCD is controlled using an HD44780-compatible 4-bit interface, which reduces the number of data pins required for the display [5]. The firmware uses this block to show the current time, date, alarm status, temperature, setting fields, and error messages.

The time block contains the external DS3231 real-time clock hardware and the firmware drivers used to access it. The DS3231 provides real-time clock data and includes battery-backed timekeeping capability, allowing the clock to maintain time information when the main system power is interrupted [3]. In the implemented firmware, the RTC is accessed through a software I2C driver on PB6 and PB7. The I2C bus uses two communication lines, SDA and SCL, for serial data transfer [4].

The same time-related subsystem also includes the non-volatile storage function for alarm settings. In the current design, alarm data is not stored in the RTC device. Instead, the firmware stores alarm settings in the internal flash memory of the STM32 through the NV storage driver. This choice keeps alarm storage under MCU firmware control and allows the settings to remain available after power loss [1], [2].

The alarm audio block provides the audible output of the system. It contains the buzzer hardware and the buzzer control function in firmware. When the current RTC time matches an enabled alarm setting, the MCU activates the buzzer output. The firmware controls the buzzer timing pattern and automatically stops the alarm after the configured ringing duration.

Overall, the block diagram shows a clear separation between power regulation, user input, central processing, timekeeping, display output, storage, and alarm audio output. This organization makes the system easier to understand, implement, test, and maintain.

Table 3.1: Main system architecture blocks and functions.

Block	Main function
Power supply	Receives the 9 V battery input, regulates the supply voltage, and indicates power status.
MCU	Executes the firmware and coordinates all input, display, RTC, storage, and buzzer

	functions.
Buttons	Provides user input for time setting, alarm setting, value increase, and value decrease.
Display	Shows the current time, date, alarm status, temperature, setting screens, and error messages.
Time / RTC	Provides real-time clock information through the DS3231 and software I2C communication.
NV storage	Stores alarm settings in STM32 internal flash memory.
Alarm audio	Generates buzzer notification when an enabled alarm time is reached.

3.2 Power Tree

The power distribution of the system is shown below:

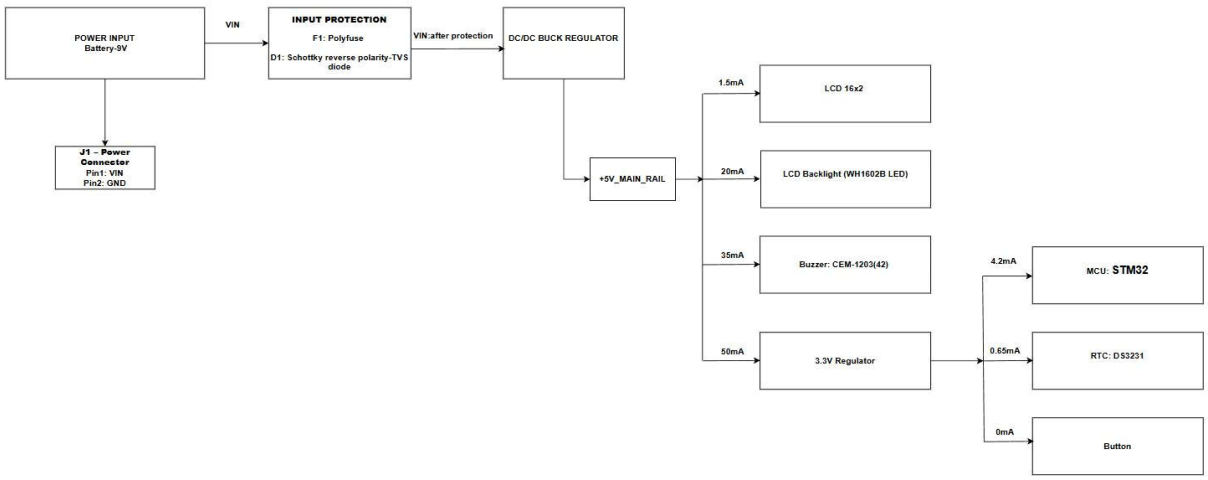


Figure 3.2: Power tree

The power distribution of the system is shown in Figure 3.2. The system uses a 9V battery as the main input source. The input voltage first passes through the input protection stage,

including a polyfuse and reverse-polarity protection diode, to reduce the risk of damage caused by overcurrent or incorrect battery connection.

After the protection stage, the voltage is converted by a DC/DC buck regulator from 9V to the main 5V rail. The +5V_MAIN_RAIL is used to supply the LCD1602 module, LCD backlight, buzzer circuit, and the input of the 3.3V regulator. These loads are placed on the 5V rail because the LCD module and buzzer are designed to operate properly at 5V.

A separate 3.3V regulator is used to generate the +3V3_MAIN_RAIL from the 5V rail. This 3.3V rail supplies the STM32 microcontroller, the DS3231 RTC module, and the button input circuit. This separation is necessary because the STM32 operates at 3.3V logic level, while the LCD and buzzer are powered from 5V.

The estimated current budget of the 5V rail includes the LCD logic current, LCD backlight current, buzzer current, and the current required by the 3.3V regulator. The 3.3V rail is designed with enough current margin for the STM32, RTC, and button inputs. This structure makes the power distribution easier to analyze and debug, since the 5V and 3.3V rails can be checked separately during testing.

4 SCHEMATIC DESIGN

This section presents the schematic design of the proposed real-time clock and alarm system, including the power supply, microcontroller, RTC module, LCD interface, button inputs, and buzzer driver circuitry.

4.1 System Schematic

The complete system schematic integrates all functional modules to form a cohesive digital clock platform. It ensures seamless communication between the microcontroller, timing peripherals, and user interface components.

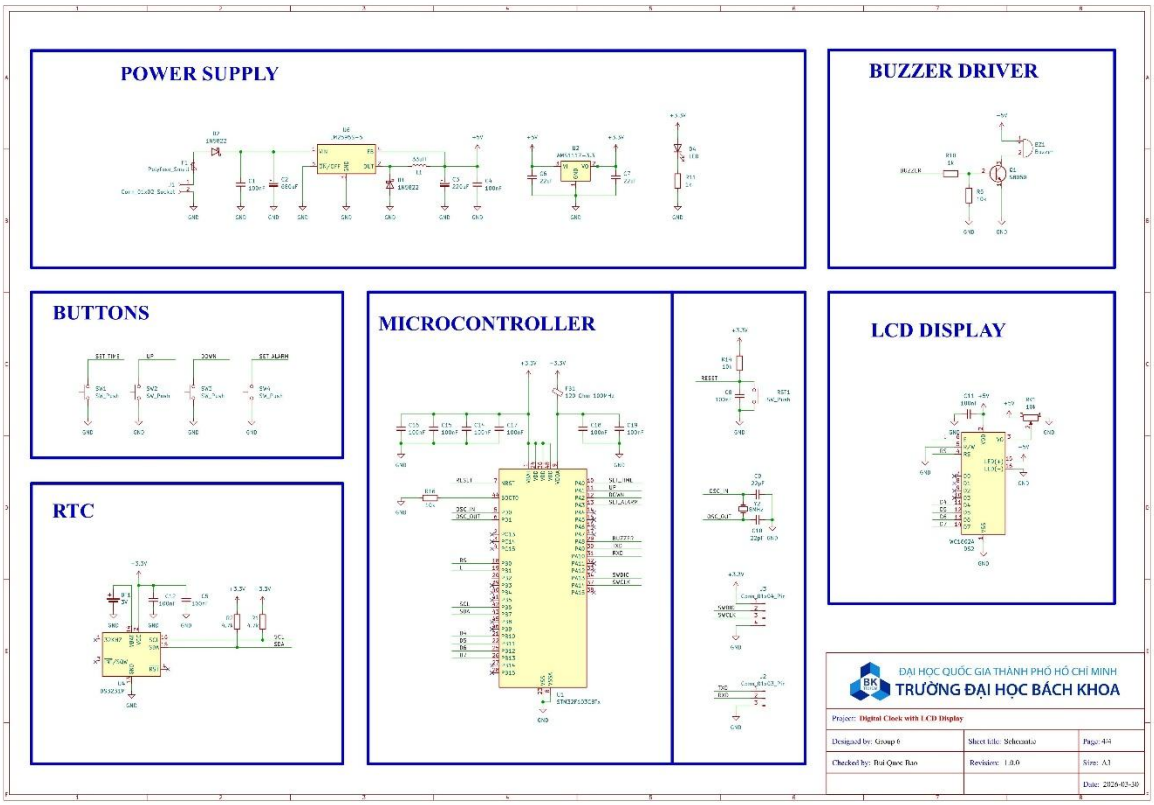


Figure 4.1: Complete System Schematic

The architecture is centered around the STM32L052C8T6 microcontroller, which manages data processing and peripheral control. The system is powered by a dual-stage regulation circuit providing 5V and 3.3V rails. Real-time data is maintained by a high-precision DS3231 RTC module via the I2C interface, while visual output is handled by a 16x2 LCD display operating in 4-bit mode.

User input is captured through a dedicated Button Interface, and system alerts are managed by a transistor-driven Buzzer circuit. The design also incorporates essential support circuitry, including a crystal oscillator for the MCU clock, a hardware reset circuit, and multiple decoupling capacitors strategically placed across the power rails to ensure operational stability and noise immunity.

4.2 Power Supply

The power supply circuit provides stable 5 V and 3.3 V output voltages for the entire system. A 9 V DC input source is used as the main power input for the embedded platform.

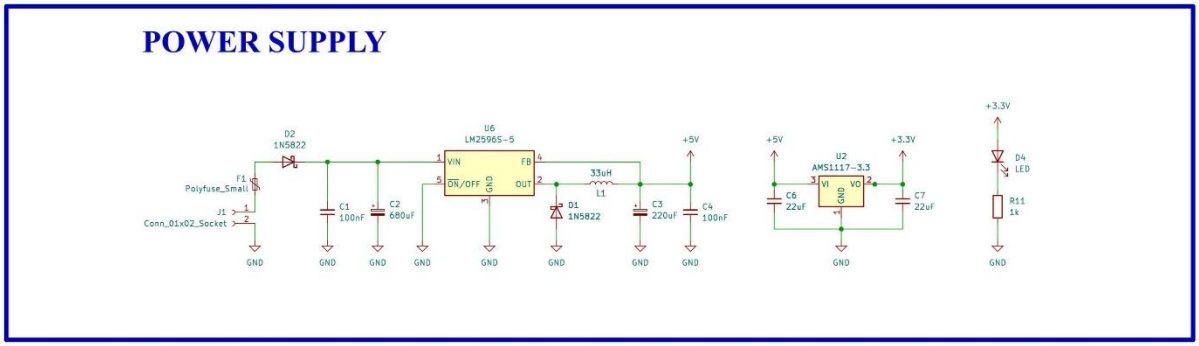


Figure 4.2: Power Supply Schematic

The input stage includes a polyfuse and a Schottky diode for overcurrent and reverse-polarity protection. An LM2596S switching regulator is used to step the input voltage down to 5 V with improved power efficiency. The regulator output is filtered using capacitors and an inductor to reduce voltage ripple and improve stability.

A secondary AMS1117-3.3 linear regulator generates the 3.3 V supply required by the STM32 microcontroller and RTC module. In addition, multiple decoupling capacitors are placed near the regulators to suppress noise and stabilize the power rails during operation. A power indicator LED is also included to provide visual confirmation of system power status.

4.3 Microcontroller

The microcontroller circuit is centered around the STM32F103C8T6, which serves as the main controller of the system. The circuit includes the minimum supporting components required for stable operation, programming, and clock generation.

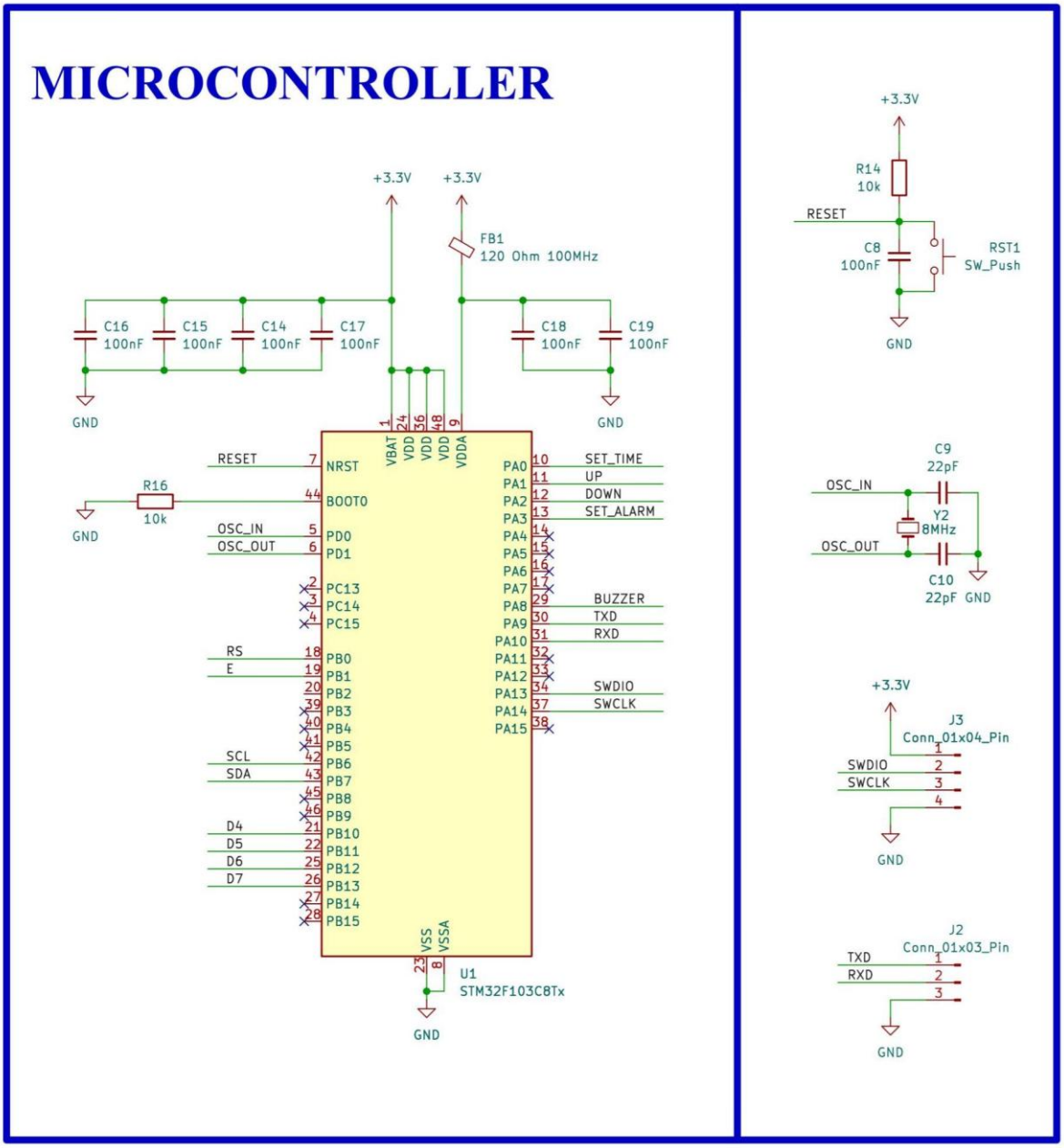


Figure 4.3: Microcontroller Schematic

The STM32F103C8T6 operates at 3.3 V and is powered through multiple decoupling capacitors placed near the supply pins to improve voltage stability and reduce noise. An external 8 MHz crystal oscillator with two 22 pF capacitors is used to provide the system clock source. The reset circuit consists of a push button and pull-up resistor connected to the NRST pin, allowing manual system reset during testing and debugging.

The BOOT0 pin is connected to ground through a pull-down resistor to ensure normal boot operation from internal Flash memory. In addition, SWD programming pins are included for

firmware downloading and debugging using an ST-Link programmer. UART pins are also exposed through a connector to support serial communication and debugging during development.

4.4 RTC

The RTC circuit is based on the DS3231M real-time clock module, which provides accurate timekeeping and alarm functionality for the system. The RTC communicates with the STM32F103C8T6 microcontroller through the I2C communication protocol.

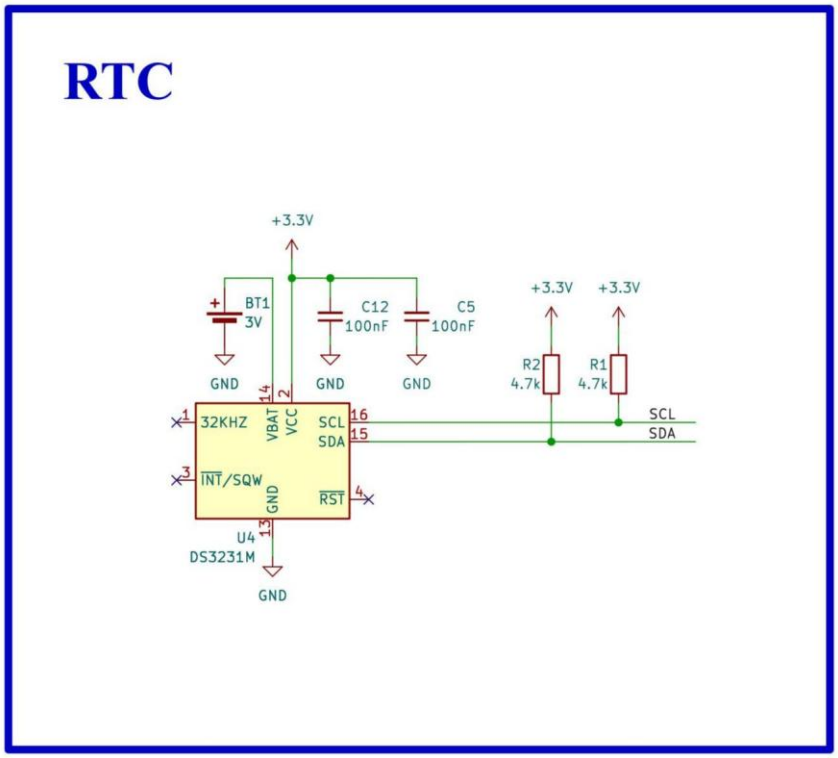


Figure 4.4: RTC Schematic

The DS3231M operates at 3.3 V and includes decoupling capacitors to improve power stability and reduce noise. A 3 V backup battery is connected to the VBAT pin to maintain timekeeping operation when the main power supply is disconnected. The SDA and SCL communication lines are connected to the microcontroller through 4.7 kΩ pull-up resistors, which are required for proper I2C bus operation.

The INT/SQW and 32 kHz output pins are not utilized in this project, as the RTC is primarily used for time and alarm management through standard I2C communication.

4.5 Button Interface

The button interface circuit allows the user to interact with the system by triggering specific functions such as setting time or alarms. This design utilizes four tactile switches configured in an **active-low** setup.

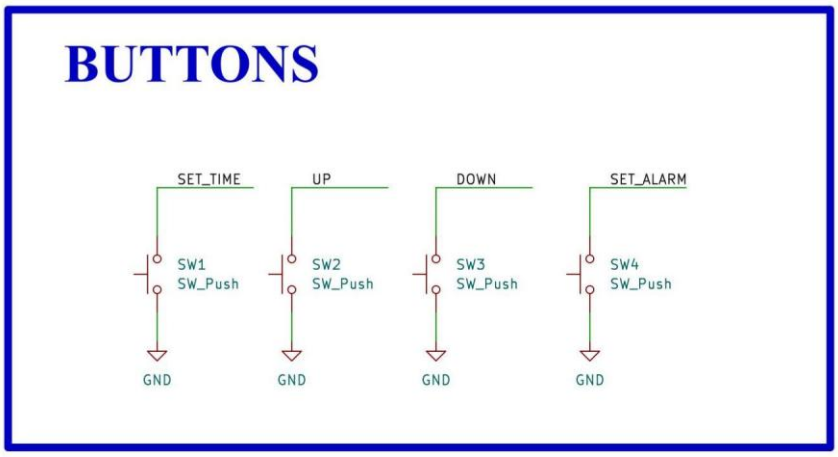


Figure 4.5: Button Interface Schematic

The circuit consists of four push buttons: **SET_TIME**, **UP**, **DOWN**, and **SET_ALARM**. Each switch connects a microcontroller input pin directly to the ground (**GND**) when pressed. This simple configuration relies on the internal pull-up resistors of the microcontroller to maintain a logic HIGH state when the buttons are idle, ensuring a stable signal without the need for external components.

The software utilizes these inputs to navigate menus and adjust system parameters, providing a responsive and intuitive user interface.

4.6 LCD Display

The LCD interface circuit uses a 16×2 character LCD module to display the current time, alarm settings, and system status information. The display is connected to the STM32F103C8T6 microcontroller using 4-bit mode to reduce GPIO usage.

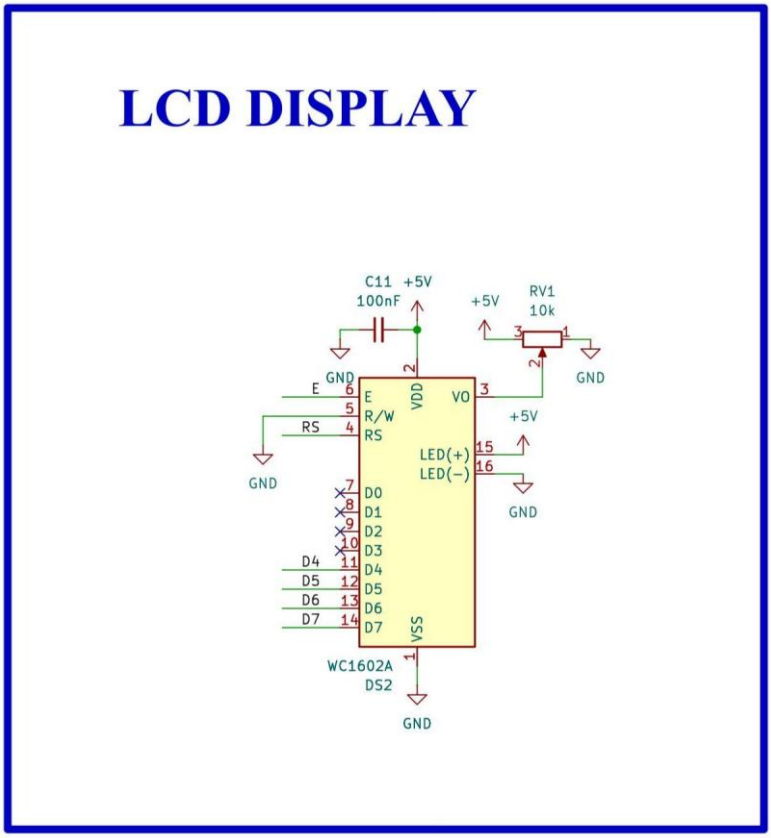


Figure 4.6: LCD Display Schematic

The LCD module operates at 5 V and uses four data lines (D4–D7) together with the RS and E control signals for communication with the microcontroller. The R/W pin is permanently connected to ground, configuring the LCD for write-only operation. A 10 kΩ potentiometer is connected to the VO pin to adjust the display contrast manually.

In addition, a decoupling capacitor is placed near the power supply pins to improve voltage stability during operation. The LCD backlight is powered directly from the 5 V supply to provide clear visual output for the user.

4.7 Buzzer Driver

The buzzer driver circuit serves as an audible notification system for alarms and user feedback. It is designed to allow a low-current microcontroller signal to drive a higher-current electromagnetic buzzer.

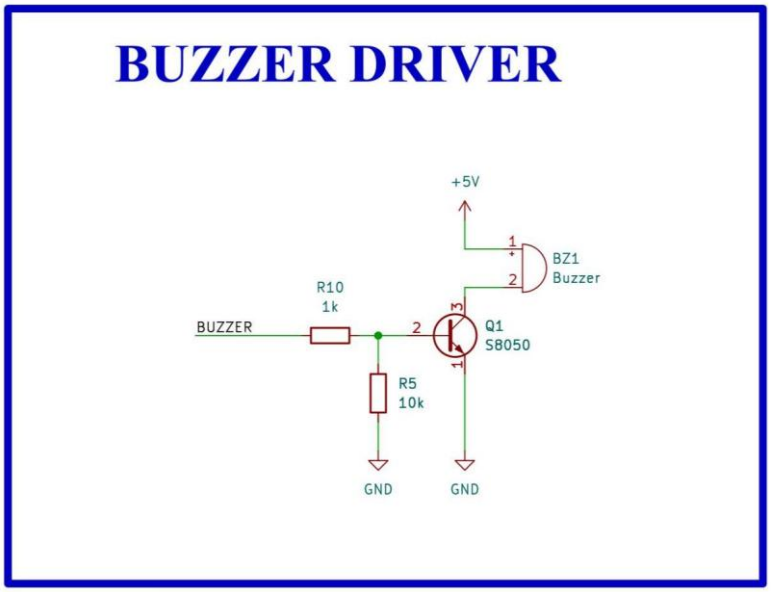


Figure 4.7: Buzzer Driver Schematic

The circuit utilizes an S8050 NPN transistor (Q1) acting as a switch. A 1kΩ base resistor (R10) limits the current from the microcontroller to the transistor's base, while a 10kΩ pull-down resistor (R5) ensures the buzzer remains off when the control pin is in a floating or high-impedance state.

When the "BUZZER" signal is driven HIGH, the transistor enters saturation, completing the path to ground and activating the 5V buzzer. This configuration protects the microcontroller pins from the inductive loads and current demands of the buzzer.

5 FIRMWARE DESIGN

5.1 Overview

The firmware is the main control part of the digital clock and alarm system. It manages user input, communicates with external devices, updates the LCD, controls the buzzer, and stores alarm settings in non-volatile memory. The firmware was developed for the STM32F103C8T6 microcontroller using STM32CubeIDE and the STM32 HAL library.

A modular firmware structure was used to improve readability, debugging, and maintenance. Each module has a clear role, such as button scanning, LCD control, RTC communication, application logic, or flash-based alarm storage. This structure makes the program easier to understand and easier to extend.

5.2 Development Environment and Design Approach

The firmware was implemented on the STM32F103C8T6 Blue Pill platform. The STM32F103x8/xB device family provides the GPIO, clock, memory, and peripheral resources required for this embedded system [1]. The project uses STM32CubeIDE and the STM32 HAL library for GPIO control, timing, flash access, and system initialization. The STM32F10xxx reference manual describes the internal memory and peripheral behavior used by this firmware [2].

An external DS3231 real-time clock (RTC) was selected for independent timekeeping and backup operation. The DS3231 supports I2C communication and includes a battery input to maintain timekeeping when the main power is interrupted [3]. In the final firmware, the RTC communication is implemented using software I2C on PB6 and PB7. The I2C bus uses SDA and SCL lines for serial communication [4], while the current firmware generates these signals directly by controlling GPIO pins.

The system clock is configured to run at 64 MHz in the implemented firmware. This clock configuration gives the system enough speed for responsive button handling, LCD updates, software I2C timing, and buzzer control.

5.3 Firmware Architecture

The firmware follows a modular architecture, as shown in Figure 5.1. The main program file, main.c, initializes the system and repeatedly calls the application task. The main behavior is implemented in app_clock.c, while lower-level hardware operations are separated into board support package (BSP) modules for buttons, LCD, RTC, and non-volatile storage.

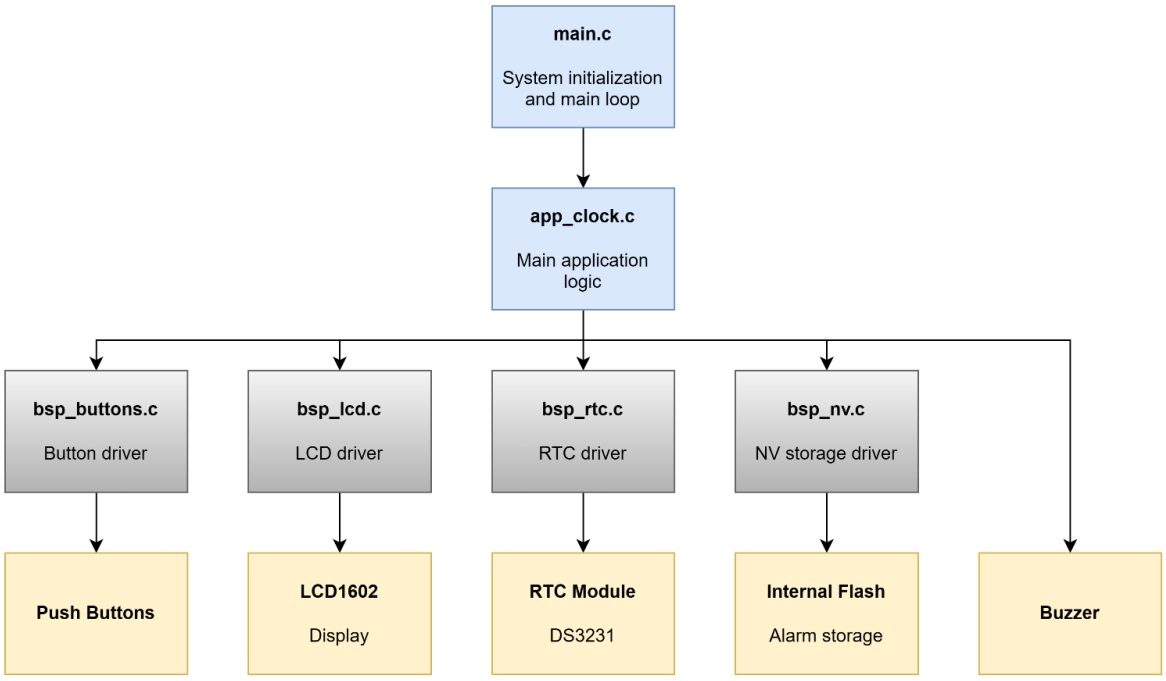


Figure 5.1: Firmware architecture of the digital clock and alarm system.

This organization separates application logic from hardware control. As a result, the firmware is easier to debug, maintain, and modify because each module has a specific responsibility.

5.4 Hardware-to-Firmware Interface

The firmware directly interacts with the hardware through the GPIO pins of the STM32F103C8T6. The final pin mapping used in the current firmware is shown in Table 5.1.

Table 5.1: Pin mapping used by the firmware

Function	STM32 Pin	Description
SET_TIME	PA0	Button for entering and stepping through time-setting mode
UP	PA1	Button for increasing the selected value
DOWN	PA2	Button for decreasing the selected value
SET_ALARM	PA3	Button for entering and stepping through alarm-setting mode
BUZZER	PA8	Output signal for buzzer control
LCD_RS	PB0	LCD register select

LCD_E	PB1	LCD enable
LCD_BACKLIGHT	PB8	LCD backlight control
LCD_D4	PB10	LCD data line D4
LCD_D5	PB11	LCD data line D5
LCD_D6	PB12	LCD data line D6
LCD_D7	PB13	LCD data line D7
RTC_SCL	PB6	Software I2C clock line
RTC_SDA	PB7	Software I2C data line

The four buttons are configured as input pull-up. Therefore, the button signal is active when the GPIO reads logic low. The buzzer and LCD signals are configured as output pins. The RTC communication lines PB6 and PB7 are reconfigured in the firmware for software I2C open-drain operation.

5.5 Main Program Flow

The file main.c is intentionally simple. It initializes the microcontroller and required firmware modules, then enters the infinite loop. The main execution flow is shown in Figure 5.2. The main flow of the firmware is as follows:

1. Initialize the HAL library
2. Configure the system clock
3. Initialize GPIO
4. Initialize the LCD driver
5. Initialize the button driver
6. Initialize the application layer
7. Enter the infinite loop and continuously call APP_CLOCK_Task().

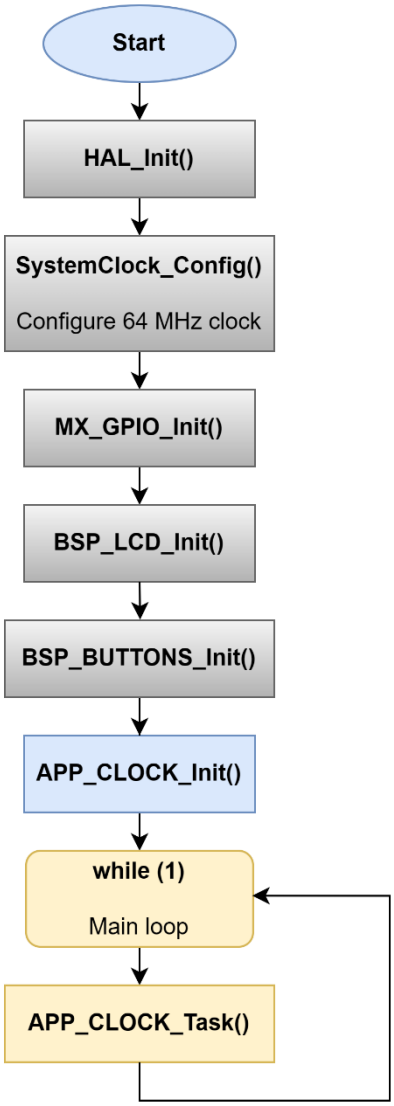


Figure 5.2: Main firmware flow from startup to continuous application execution.

After initialization, APP_CLOCK_Task() is called repeatedly. This task updates the runtime behavior of the system, including button scanning, user interface handling, RTC reading, LCD refreshing, alarm checking, and buzzer control.

5.6 Firmware Modules

5.6.1 Configuration and Shared Data Types

The file app_config.h contains the general configuration of the firmware. It defines pin mapping, timing constants, RTC address information, default RTC values, flash storage settings, and the maximum number of alarms. Keeping these values in one configuration file

improves maintainability because hardware-related and timing-related settings are grouped in one place.

Several timing constants are used for periodic tasks. For example, APP_UI_PERIOD_MS is set to 50 ms, so the LCD content is rebuilt and updated every 50 ms during runtime. This keeps the displayed clock, alarm-setting screen, blinking field, and user interface state synchronized with the internal firmware variables.

Table 5.2: Important firmware timing constants

Parameter	Value	Description
APP_UI_PERIOD_MS	50 ms	Refreshes the LCD content and user interface
APP_BLINK_PERIOD_MS	300 ms	Toggles the visible state of the selected field
APP_BTN_REPEAT_MS	100 ms	Repeats UP/DOWN adjustment while the button is held
APP_BUZZ_ON_MS	300 ms	Keeps the buzzer ON during one beep cycle
APP_BUZZ_OFF_MS	100 ms	Keeps the buzzer OFF between beep cycles
APP_BUZZ_TOTAL_MS	60000 ms	Sets the total alarm ringing duration

The file app_types.h defines the shared data structures used by the firmware. The rtc_datetime_t structure stores the current time and date, while the alarm_cfg_t structure stores the hour, minute, second, and enable status of one alarm.

5.6.2 Application Layer

The main application logic is implemented in app_clock.c. During initialization, APP_CLOCK_Init() creates LCD custom characters, displays startup messages, loads saved alarm data from flash, checks the RTC, reads the initial time, synchronizes the display/edit variables, and turns the buzzer off.

During runtime, APP_CLOCK_Task() is called repeatedly from the main loop. This function scans and debounces buttons, handles time-setting and alarm-setting logic, updates the blink state, refreshes the LCD, reads the RTC when required, and controls the alarm output. The runtime flow of this task is shown in Figure 5.3.

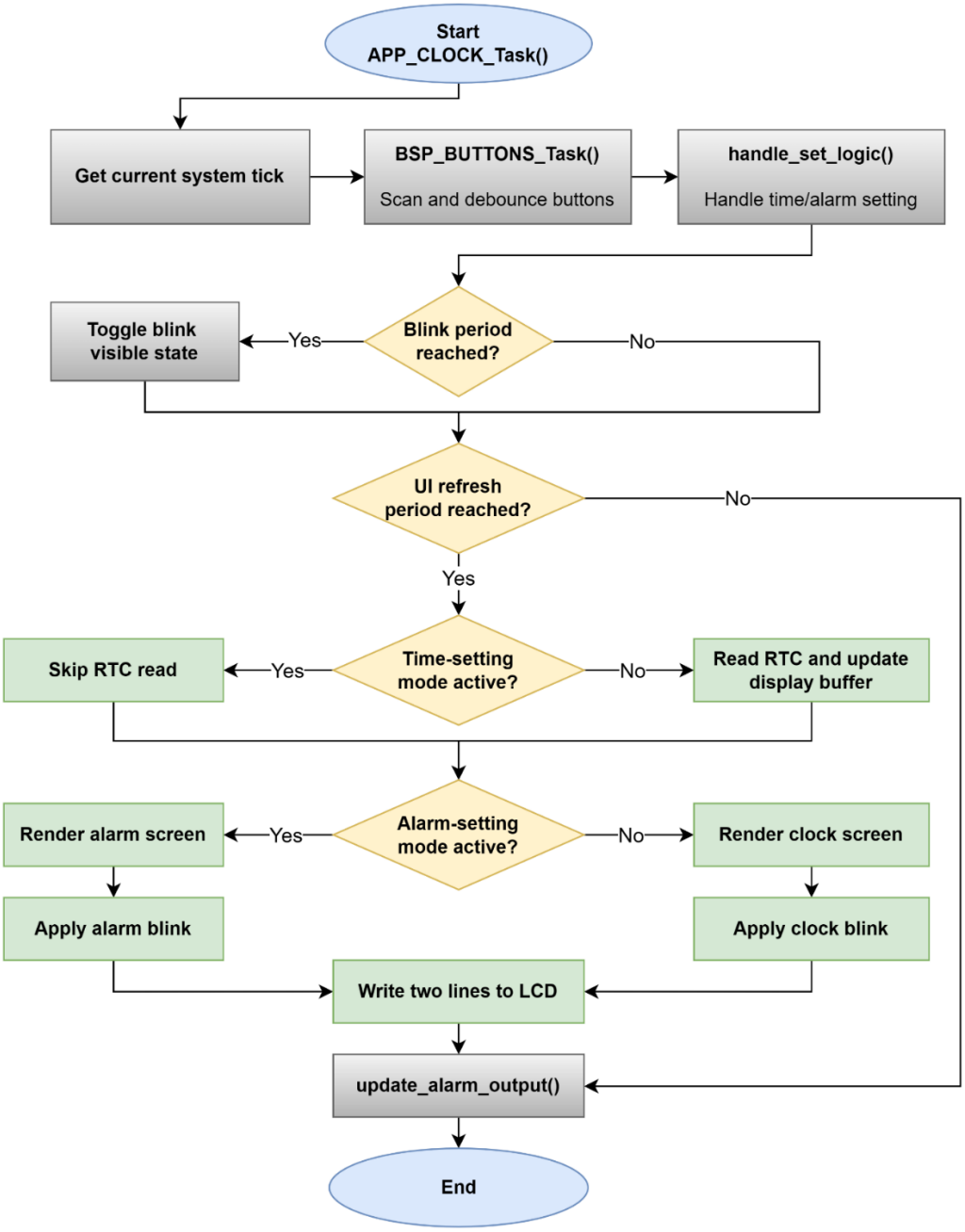


Figure 5.3: Runtime flow of the main application task APP_CLOCK_Task().

This design makes app_clock.c the main application layer, while the low-level hardware details remain inside their respective BSP modules.

5.6.3 Button Driver

The button driver in bsp_buttons.c provides stable input reading for the four push buttons. Since mechanical buttons produce bouncing during press and release, the driver applies a 10 ms debounce check before accepting a state change. It provides two types of input

information: a one-time press event and a held state. The press event is used for actions such as moving to the next field, while the held state is used for continuous UP/DOWN adjustment.

5.6.4 LCD Driver

The LCD driver in bsp_lcd.c controls the LCD1602 in 4-bit mode. The HD44780-compatible LCD interface supports data transfer using four data lines, which reduces the number of GPIO pins required for the display [5]. The driver handles LCD initialization, command/data transfer, line writing, screen clearing, and custom character creation. In this project, custom characters are used for the thermometer and alarm bell symbols.

5.6.5 RTC Driver

The RTC driver in bsp_rtc.c communicates with the DS3231 through software I2C. It generates start and stop conditions, writes and reads bytes, accesses RTC registers, and converts time values between BCD and normal integer format. The DS3231 provides real-time clock data through its I2C register interface and also provides temperature information that the firmware reads when available [3].

The function BSP_RTC_ReadDateTime() reads the current date and time from the RTC, while BSP_RTC_WriteDateTime() writes updated values back to the RTC. At startup, BSP_RTC_ProbeAndInitDefault() checks whether the RTC data is valid. If invalid values are detected, the firmware writes the default date and time defined in the source code.

5.6.6 Non-Volatile Storage Driver

The non-volatile storage driver in bsp_nv.c stores alarm settings in the internal flash memory of the STM32. The STM32F103x8/xB device includes embedded flash memory [1], and the STM32F10xxx reference manual describes the flash memory interface and programming behavior [2]. In this project, the last 1 KB flash page is reserved for alarm storage.

Instead of repeatedly saving alarm data at one fixed address, the firmware uses a slot-based log structure. Each save operation writes a new slot containing a magic value, sequence number, version, alarm data, checksum, and commit marker. The sequence number identifies the latest valid record, the checksum validates data integrity, and the commit marker is written last to confirm that the slot has been completely written. The flash memory organization is shown in Figure 5.4.

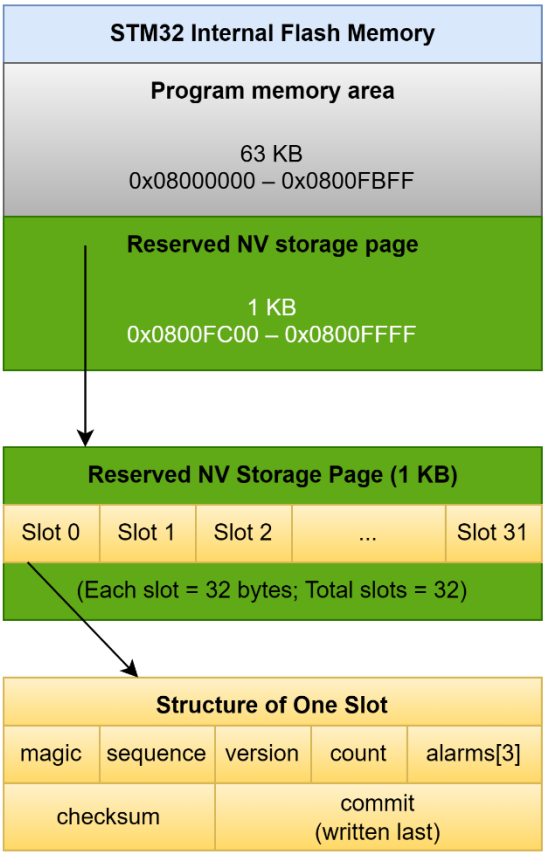


Figure 5.4: Flash-based non-volatile storage structure for alarm settings.

When alarms are saved, BSP_NV_SaveAlarms() validates the data, finds an empty slot, writes the new record, writes the commit marker last, and verifies the stored data. During startup, BSP_NV_LoadAlarms() scans the storage page, loads the latest valid alarm record, or falls back to default alarm values when no valid record is found.

5.7 User Interface and Operating Modes

The firmware provides three main user interface states: normal clock display, time-setting mode, and alarm-setting mode. In normal display mode, the LCD shows the current time, date, day-of-week, alarm status icon, and RTC temperature when available.

In time-setting mode, the SET_TIME button moves through the editable fields: hour, minute, second, day-of-week, date, month, year, and finish. The selected field blinks, and the UP/DOWN buttons adjust its value. When the final step is reached, the firmware writes the new date and time to the RTC.

In alarm-setting mode, the SET_ALARM button moves through alarm index, enable/disable status, hour, minute, second, and finish. The UP/DOWN buttons modify the selected field. When the final step is reached, all alarms are saved to flash and verified by reading the data back. This process is shown in Figure 5.5.

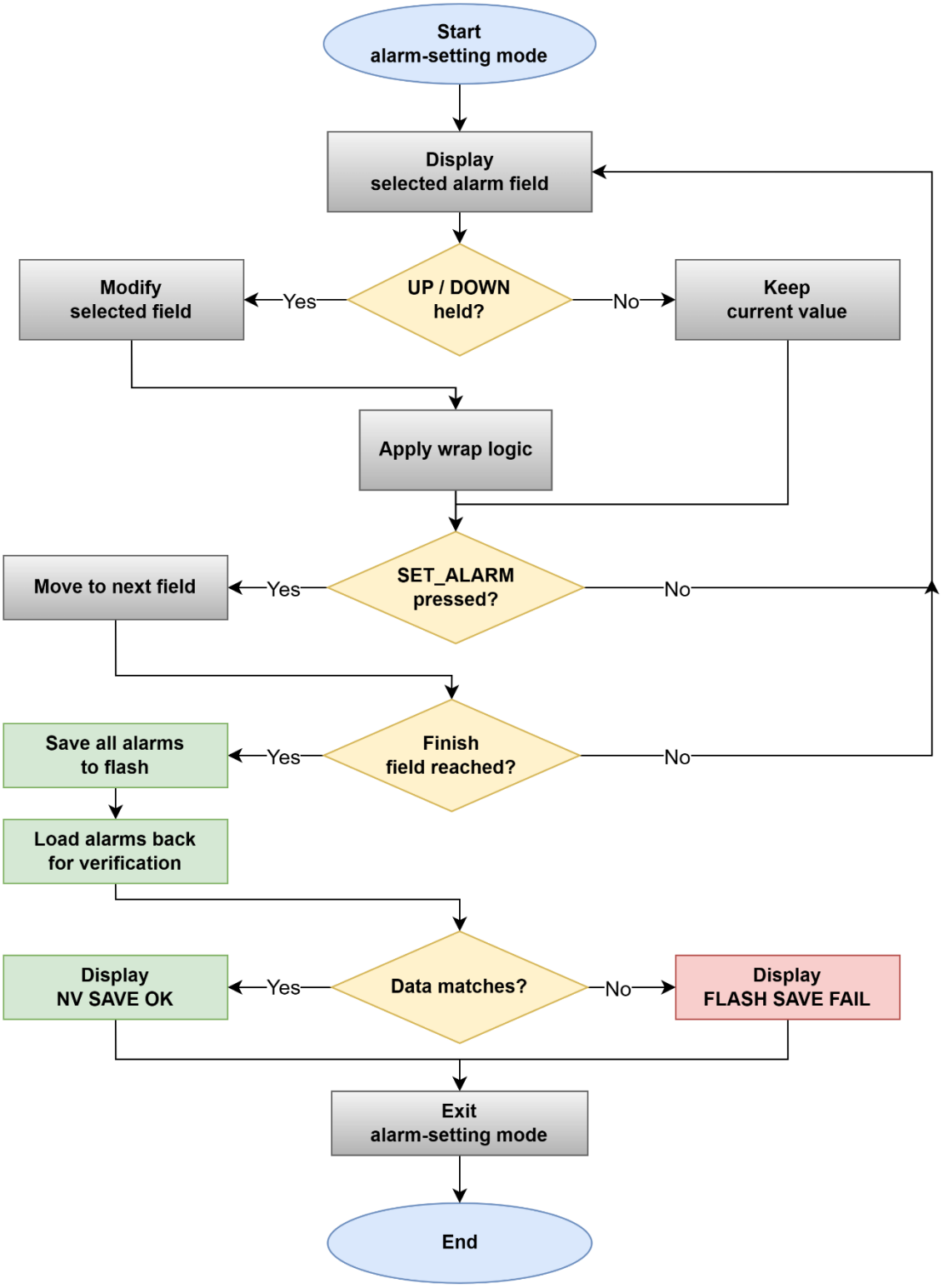


Figure 5.5: Alarm-setting procedure and flash save verification.

The firmware uses wrap-around adjustment to improve usability. For example, hours wrap between 0 and 23, minutes and seconds wrap between 0 and 59, months wrap between 1 and

12, and the alarm index wraps between alarm 1 and alarm 3. This allows the user to continue pressing UP or DOWN without being blocked at the boundary value.

5.8 Alarm Management Logic

The alarm logic is implemented in app_clock.c. During runtime, update_alarm_output() compares the current time with all enabled alarms. When the hour, minute, and second match an enabled alarm, the buzzer is activated.

To avoid repeated triggering during the same second, the firmware uses an alarm signature based on the current date and time. The buzzer follows a 300 ms ON and 100 ms OFF pattern and stops automatically after 60 seconds. The alarm output is also stopped when the user is editing the time, editing alarms, or when no alarm is enabled.

5.9 Reliability and Error Handling

The firmware includes several reliability mechanisms. During startup, the RTC is checked before normal operation. If the RTC cannot be accessed, the LCD displays “RTC READ ERROR” and “CHK SDA/SCL/RTC”. If invalid RTC values are detected, the firmware writes default date and time values to the RTC.

For alarm storage, the flash record is validated using a magic value, version field, sequence number, checksum, and commit marker. After saving alarm settings, the firmware reads the data back and compares it with the original alarm data in RAM. If the values match, the save operation is accepted; otherwise, an error message is displayed. If no valid alarm data is found during startup, default alarms are loaded so the system can continue operating.

5.10 Firmware Validation

The firmware was validated through functional testing on the STM32F103C8T6 platform. The tests covered system startup, RTC communication, time setting, alarm setting, flash persistence, alarm triggering, buzzer duration, button response, and LCD update behavior. The results confirmed that the system initializes correctly, reads and displays RTC data, stores alarm settings after power cycling, triggers the buzzer at the configured alarm time, and provides stable button and display operation.

5.11 Summary of Firmware Section

In summary, the firmware was implemented with a modular structure that integrates button input, LCD output, RTC communication, alarm handling, buzzer control, and flash-based alarm storage. The design uses software debounce, software I2C, display refresh timing, alarm verification, and safe default behavior to improve usability and reliability.

6 IMPLEMENTATION

6.1 Breadboard Setup

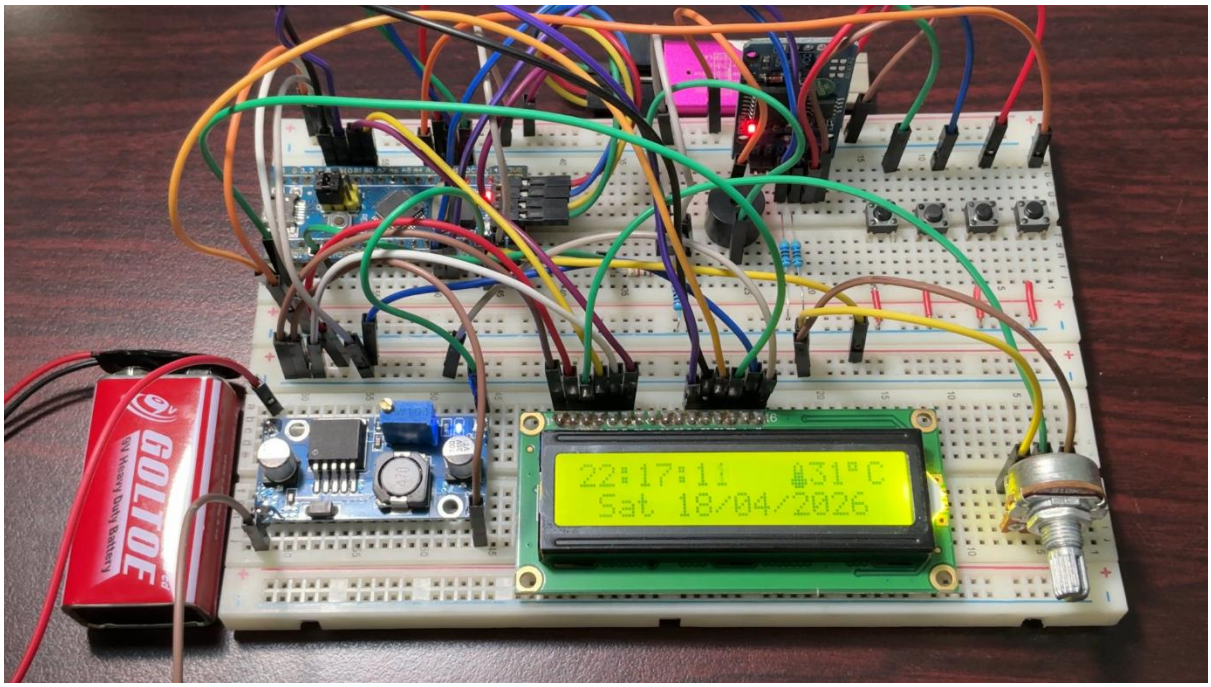


Figure 6.1: Breadboard Setup

The first working version of the system was implemented on a breadboard. This step was necessary before PCB-level development because it allowed the group to verify the schematic connections using real components.

The breadboard prototype included the STM32 board, RTC module, LCD1602 display, push buttons, buzzer, and power connections. The RTC module was connected to the microcontroller through I2C communication, while the LCD and buttons were handled through GPIO pins.

This setup helped the group test the firmware directly with the hardware. The LCD output, button response, RTC reading, time setting, and buzzer alarm were checked during this stage. Several debugging activities were also performed on the breadboard, especially for wiring, button response, and alarm storage behavior.

However, the breadboard implementation also has limitations. Jumper wires can become loose, contact resistance may affect stability, and the layout is not as reliable as a soldered

PCB. For this reason, the breadboard was used mainly as a functional prototype rather than a final hardware platform.

6.2 PCB Design

This section details the physical implementation of the digital clock circuit onto a printed circuit board. The design was developed using a 2-layer FR4 approach, focusing on signal integrity, user ergonomics, and noise reduction.

6.2.1 Layout and Placement Strategy

The PCB measures **129.05 x 95.70 mm**. Components are organized into functional groups to streamline signal flow and reduce electromagnetic interference (EMI).

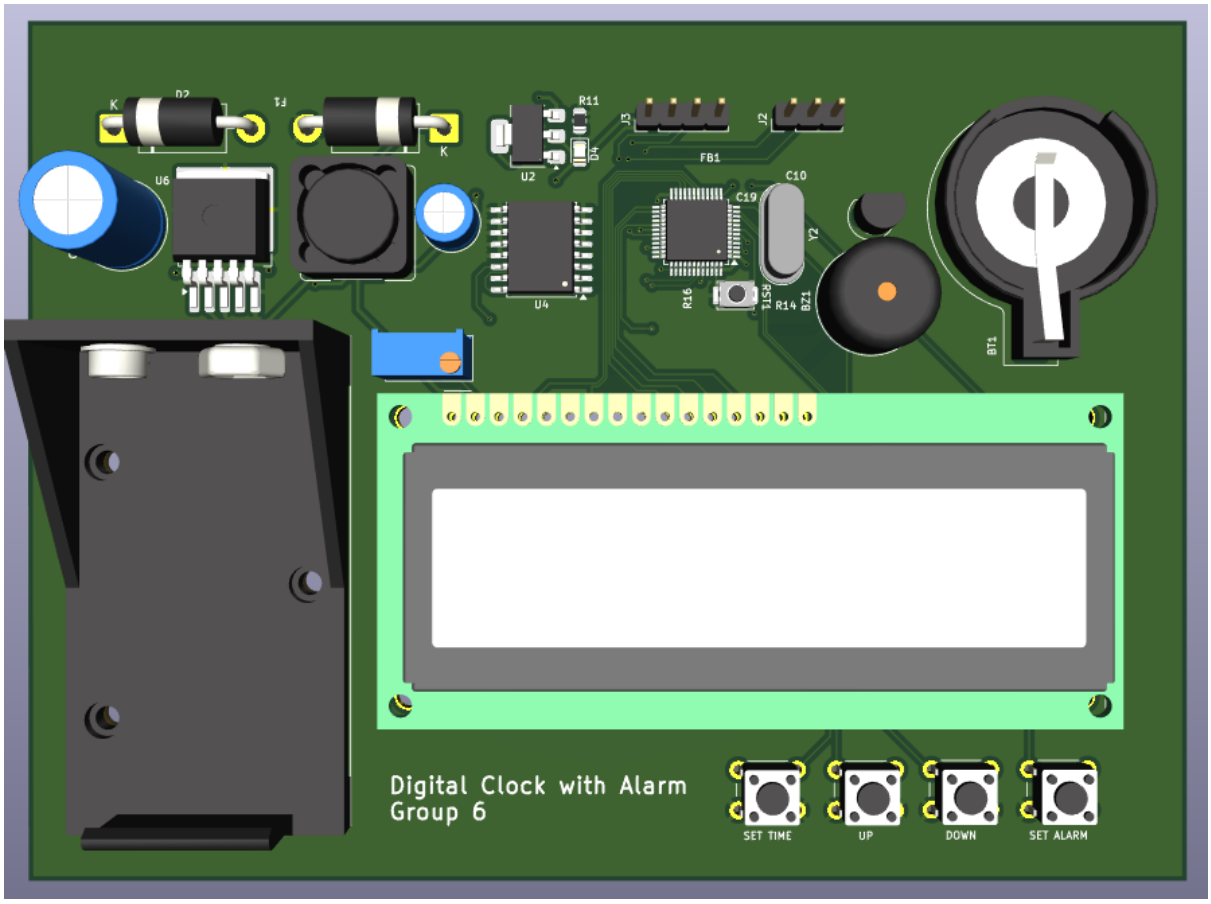


Figure 6.2: PCB 3D front view

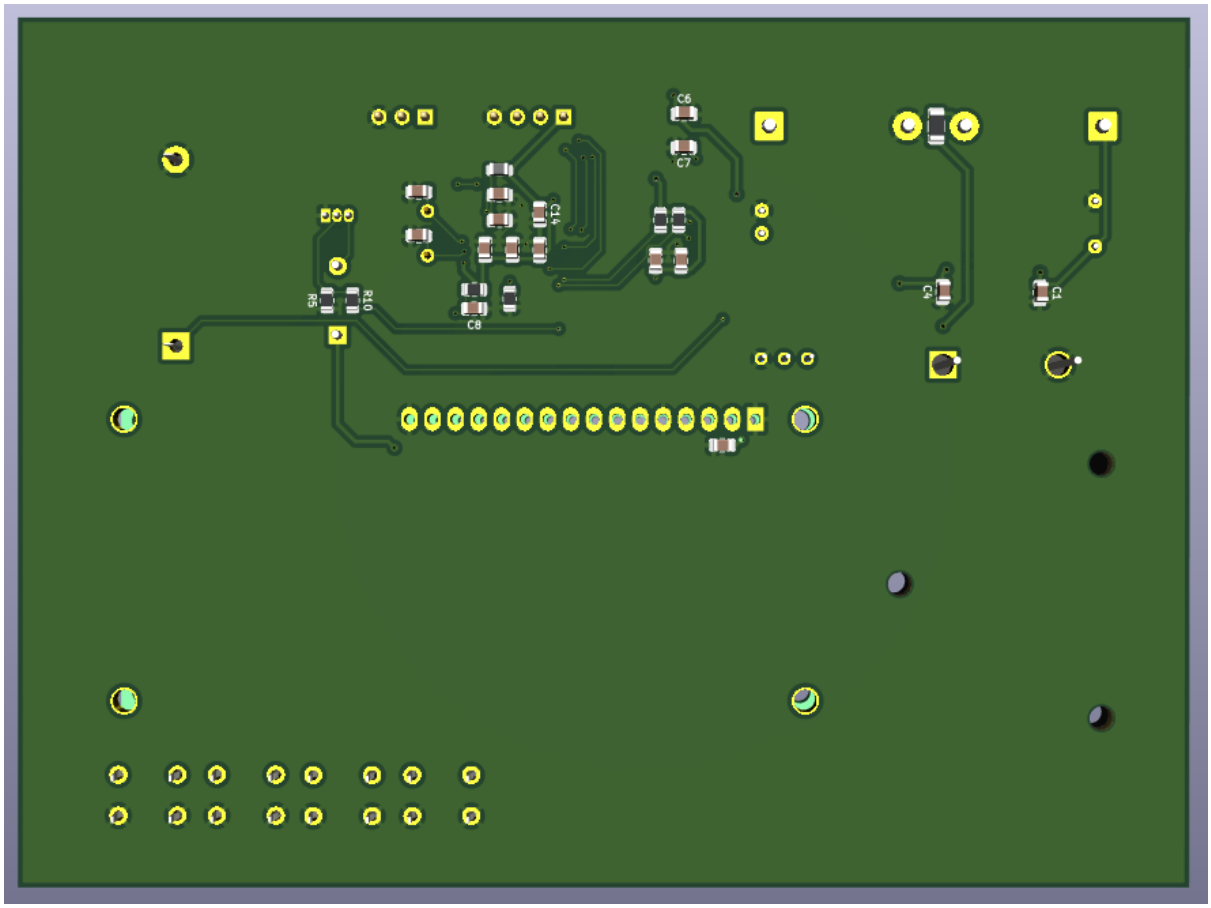


Figure 6.3: PCB 3D back view

- **Functional Grouping:** The power management section is isolated on the far left to keep high-frequency switching noise away from sensitive logic. The STM32 MCU is placed centrally to ensure optimal signal distribution to the LCD, RTC, and peripherals.
- **User Experience (UX):** The 16x2 LCD and tactile buttons are aligned along the lower half of the board for easy user interaction and clear visibility.
- **Space Optimization:** To minimize loop inductance and save top-side area, decoupling capacitors are placed on the bottom layer, positioned directly beneath the MCU power pins.

6.2.2 Routing Strategy and Technical Specifications

Routing was performed with a clear distinction between power and signal nets to ensure stable operation and easy fabrication.

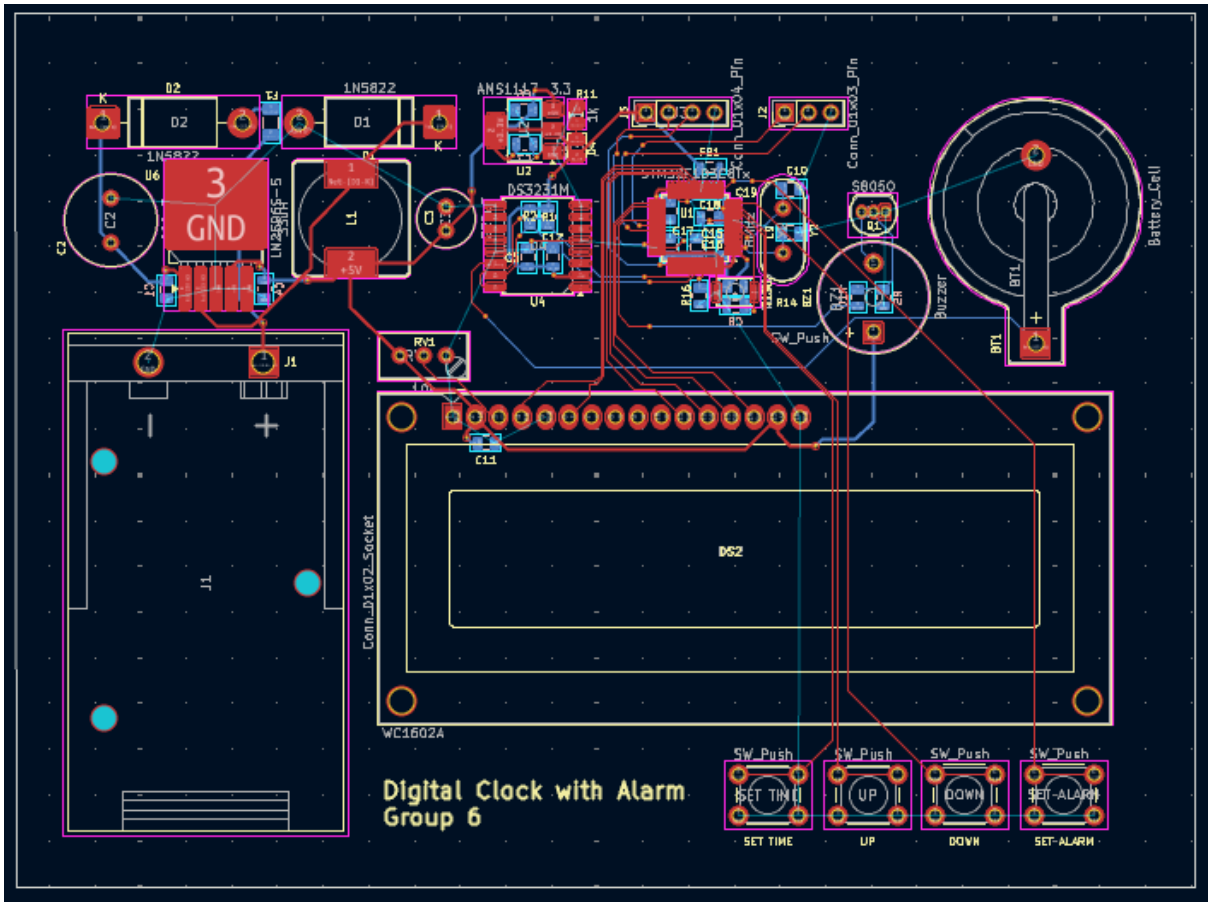


Figure 6.4: PCB 2D view

Net Type	Track Width	Via (Size/Hole)
Power (5V, 3.3V)	0.4 mm	0.8 / 0.4 mm
Signal / Data	0.2 mm	0.6 / 0.3 mm

The design utilizes a 2-layer architecture. Power traces are kept as wide as possible (0.4 mm) to minimize impedance, while signal traces use a 0.2 mm width to navigate the dense routing around the MCU. 45-degree trace angles are used throughout to maintain signal integrity and prevent acid traps during potential manufacturing.

6.2.3 Design Considerations and Signal Integrity

Special attention was paid to noise suppression and thermal stability:

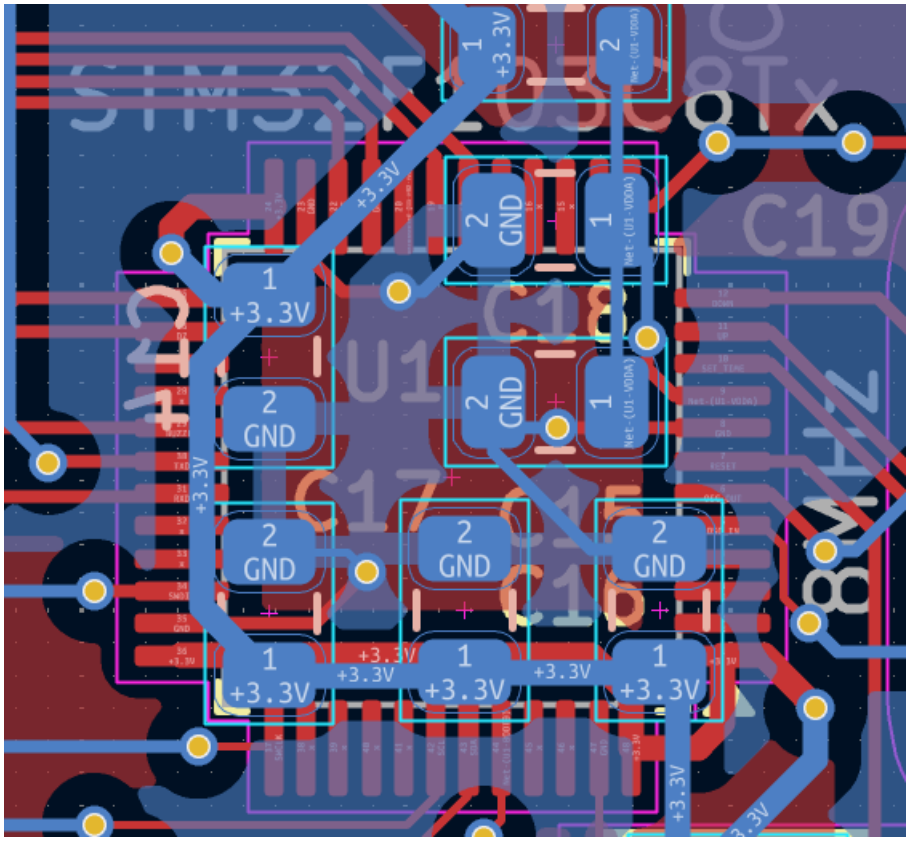


Figure 6.5: Decoupling capacitors are placed on the back layer

- **Decoupling:** High-frequency ceramic capacitors are placed as close as possible to the VDD/VSS pins of the STM32 and DS3231.
- **Ground Plane:** A solid ground plane is used on the bottom layer to provide a low-impedance return path for signals and to help dissipate heat from the regulators.
- **Trace Isolation:** Critical timing signals for the RTC and MCU crystal are kept short and isolated from high-current paths.

6.2.4 Design Verification and Results

Although the board was not physically fabricated, the design underwent a rigorous Design Rule Check (DRC) based on standard manufacturing constraints (6 mil clearance/trace).

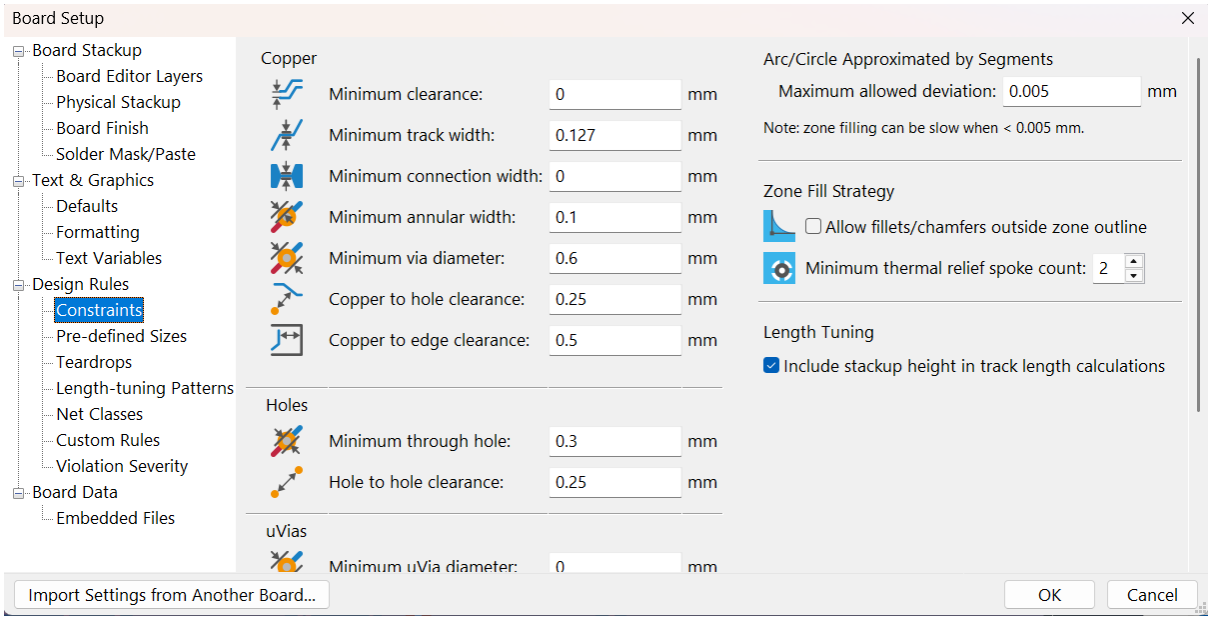


Figure 6.6: PCB Design Rules Constraints

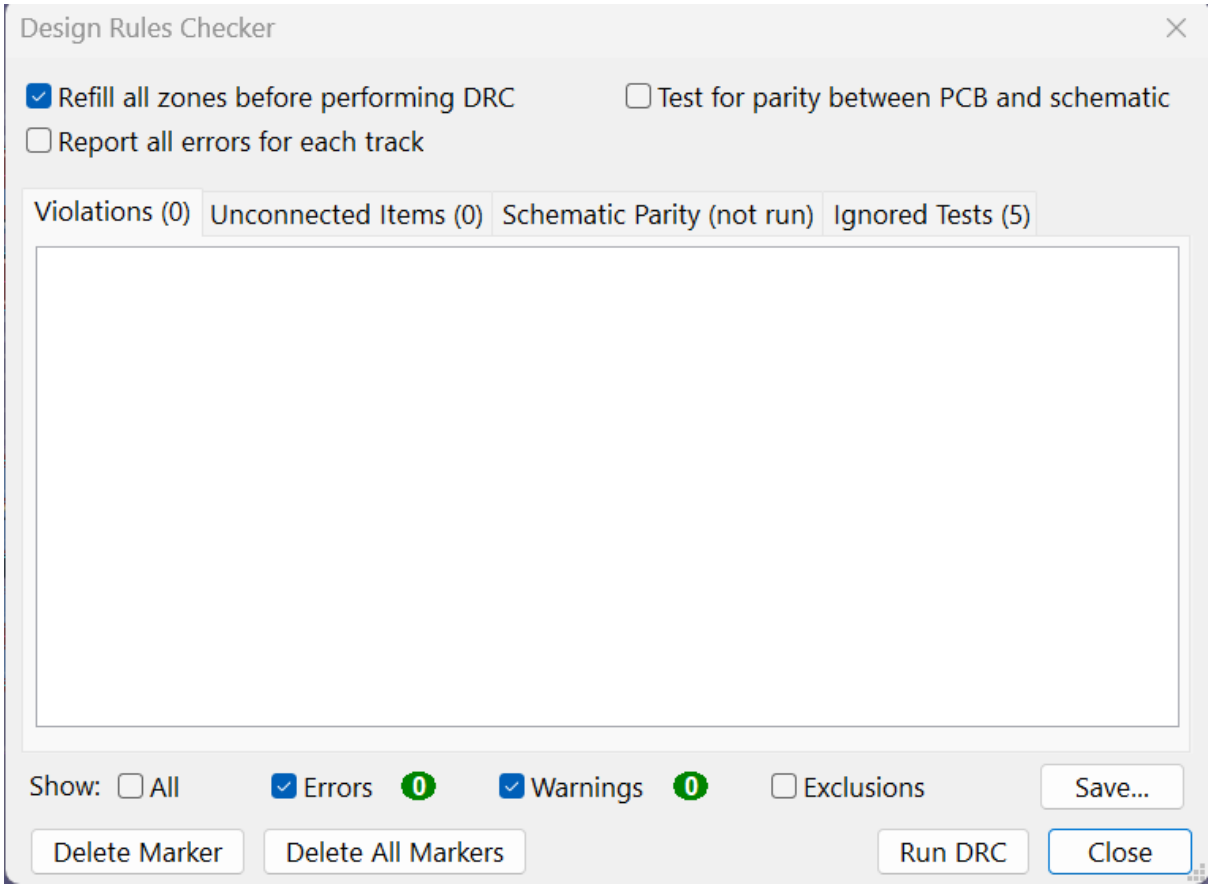


Figure 6.7: Design Rules Checker Results

The design successfully passed all DRC tests with zero errors and zero warnings, confirming that the layout is fully compatible with professional PCB fabrication services. The 3D visualization confirms the mechanical clearance of all components, including the battery holder and LCD headers.

7 RESULTS & DISCUSSION

7.1 Functional Verification

Table 7.1: Functional Verification

Function	Verification Method	Status	Discussion
Time and date display	Observe LCD output during operation	Passed	The LCD displays time and date clearly with second-level update.
RTC communication	Read time data from RTC module	Passed	The system can read real-time data from the RTC module.
Time setting	Set system date/time to 18/04/2026	Passed	The updated value is displayed correctly after setting.
Button input	Use push buttons to adjust time and alarm	Passed	Buttons respond to user input during setting modes.
Alarm triggering	Set alarm time and wait for matching time	Passed	The buzzer activates when the alarm condition is reached.
Buzzer output	Observe alarm sound	Passed	The buzzer produces a clear audible notification.
Three alarm non-volatile storage	Configure three alarm slots, power-cycle the system, and reload the settings	Passed	All three alarm configurations were preserved in non-volatile memory and restored correctly after restart.
Alarm storage	Reset/power cycle after saving alarms	Passed	Three alarm values can be stored and retained after

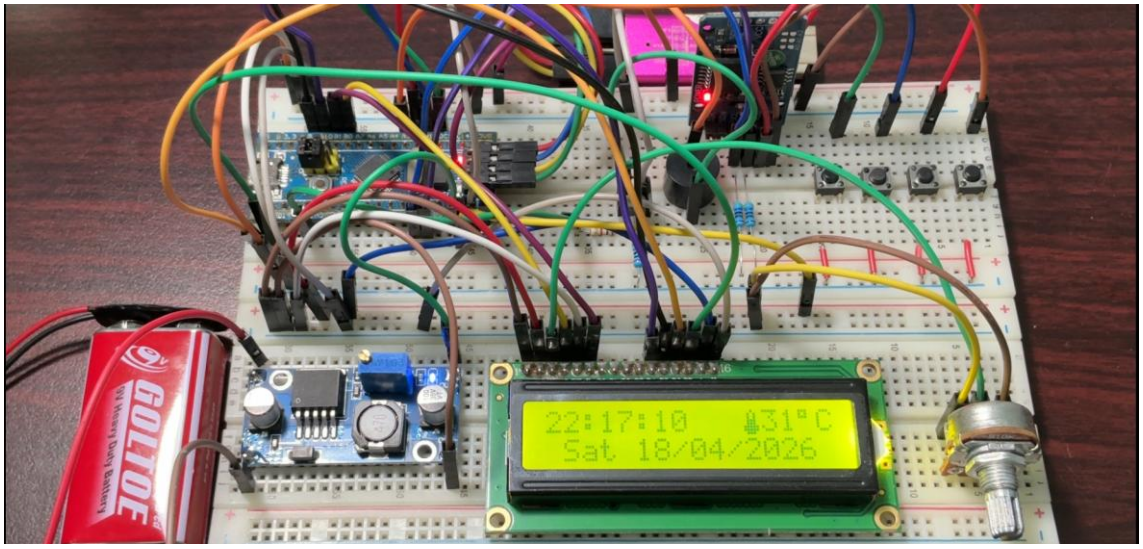
			reset/power loss.
PCB design check	Run design rule check in PCB software	Passed	PCB layout was checked successfully, but not fabricated.

The system was verified based on the main functions required for the digital clock with alarm. The verification was performed on the breadboard prototype using the actual STM32 board, RTC module, LCD1602, push buttons, and buzzer.

The RTC reading and LCD display were tested first. The LCD showed the current time and date clearly, and the seconds were updated continuously during operation. The time-setting function was also verified by setting the date to 18/04/2026, then checking whether the updated value continued to be displayed correctly.

The alarm function was tested in two stages. First, a single alarm was configured and checked by waiting for the current time to match the alarm time. When the match occurred, the buzzer produced a clear audible sound. Second, the improved firmware was tested with three alarm slots. The system was able to store three different alarm settings in non-volatile memory. After reset or power loss, all three alarm values were loaded back correctly, confirming that the alarm storage function worked as expected.

7.2 System Demonstration



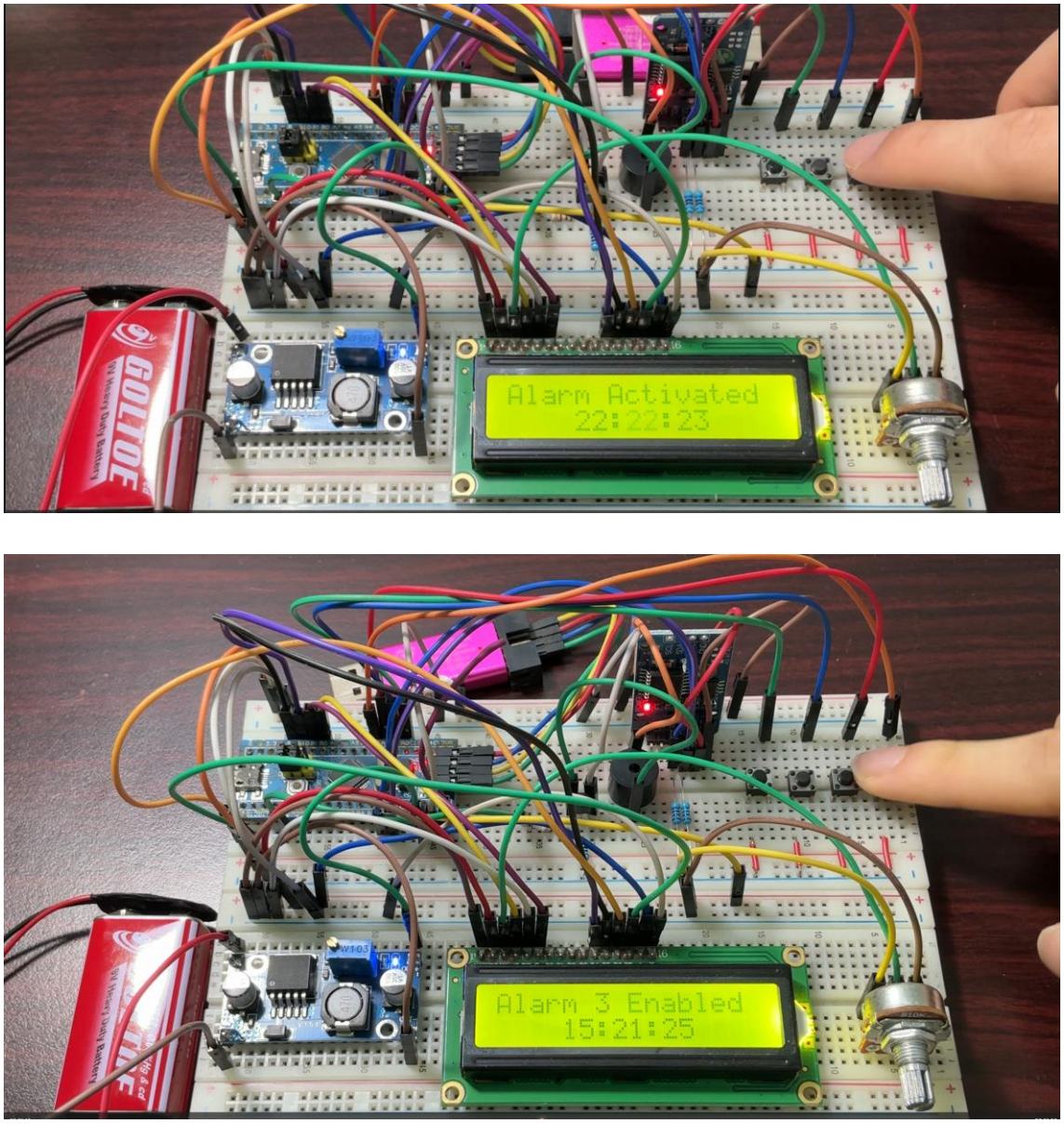


Figure 7.1: Breadboard Demonstration

The working prototype was demonstrated on the breadboard setup. During the demo, the LCD1602 showed the current time and date clearly, with the time updating down to the second. This confirmed that the RTC reading and LCD refresh logic were working properly.

The time-setting function was tested by setting the system date to 18/04/2026. After the setting process was completed, the LCD continued to display the updated date and time correctly.

The alarm function was also tested in figure 11.2. After an alarm value was configured, the system compared the current time with the stored alarm values. When the alarm condition

was reached, the buzzer generated a clear audible sound. This demonstrated that the alarm logic, buzzer control, and stored alarm settings were working together correctly.

In addition to the basic alarm trigger, the system was also tested with multiple alarm settings. Three alarm slots were configured and saved. After the system was reset or powered off and then turned on again, the alarm values were still retained. This result confirmed that the alarm configuration was not only stored temporarily in RAM, but also preserved in non-volatile memory.

The demonstration showed that the prototype achieved the core functions expected from a digital alarm clock: real-time display, user setting, alarm triggering, and alarm storage.

7.3 Limitations

Although the prototype achieved the main functional goals, several limitations still remain.

First, the system was tested mainly on a breadboard. This is suitable for prototyping and debugging, but it is not ideal for long-term operation because jumper wires and loose contacts may reduce stability.

Second, the PCB was only designed and checked by DRC. Since it was not fabricated, the group has not verified the circuit on a real PCB. Therefore, PCB-level reliability, soldering quality, and actual board behavior are still future work.

Third, the firmware still needs improvement in date validation. The current implementation should be extended to handle different month lengths and leap-year conditions correctly. For example, February should have 28 days in a normal year and 29 days in a leap year.

Finally, the user interface can be improved. Possible improvements include clearer setting-mode feedback, better button response, and multiple alarm sound patterns.

8 CONCLUSION

This project implemented a functional digital clock with alarm using an STM32 microcontroller, RTC module, LCD1602 display, push buttons, and buzzer. The system was able to display real-time data, allow user configuration, trigger alarms, and store up to three alarm settings. These alarm settings were retained after reset or power loss, which confirms that the non-volatile storage mechanism worked correctly.

The breadboard prototype confirmed that the main hardware and firmware modules were integrated successfully. The group also completed the schematic and PCB layout, and the PCB design passed design-rule checking. However, the PCB was not fabricated in this phase, so the final hardware validation was limited to breadboard testing.

Through this project, the group gained practical experience in embedded system integration, GPIO control, RTC communication, LCD display handling, button-based user input, alarm logic, and non-volatile data storage.

Future improvements should focus on firmware reliability and user experience. In particular, the date validation logic should be improved to support month length and leap-year handling. The prototype can also be refined by improving wiring stability, adding more alarm sound patterns, and eventually fabricating the PCB for a more compact and reliable hardware version.

9 REFERENCE

[1] STMicroelectronics, STM32F103x8, STM32F103xB datasheet (DS5319), Rev. 20, 2025. Available: https://www.st.com/resource/en/datasheet/stm32f103c8.pdf

[2] STMicroelectronics, RM0008 reference manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced Arm-based 32-bit MCUs, Rev. 21, 2021. Available: https://www.st.com/resource/en/reference_manual/rm0008-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

[3] Analog Devices, DS3231 extremely accurate I2C-integrated RTC/TCXO/crystal datasheet, 2015. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf

[4] NXP Semiconductors, UM10204 I2C-bus specification and user manual, Rev. 7.0, 2021. Available: https://www.nxp.com/documents/user_manual/UM10204.pdf

[5] Hitachi, HD44780U (LCD-II) dot matrix liquid crystal display controller/driver datasheet, Rev. 0.0, 1999. Available: https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf